Padrões de Projeto Criacional

PANDORA | Linguagem de Programação III | 6º A | 2026

Objetivo do documento

Documentar os cinco padrões criacionais identificados na implementação do PANDORA, relacionando o problema resolvido, os participantes concretos, os papéis e as fontes de código.

Aplicação ao projeto

A construção de ordens de serviço, a escolha do processamento dos relatos, a preparação de famílias de manutenção, a clonagem de planos e o compartilhamento da configuração de IA possuem estruturas distintas. A classificação considera o comportamento e as relações entre participantes, além dos nomes das classes.

Os diagramas detalham os padrões presentes na implementação. Os contratos abstratos Python são apresentados como classes abstratas, com generalização pelos participantes concretos. As associações indicam referências mantidas e as dependências indicam uso ou criação de objetos.

Padrões de Projeto Criacional	
PANDORA - Plataforma Inteligente de Locação e Manutenção de Equipamentos com LLM	

Singleton

AISettings compartilha a configuração de IA obtida por get_instance(), com controle de concorrência por Lock. AIService, GroqProvider e AIAssistantFacade consomem essa configuração. A unicidade é por processo e pelo acesso ao método; o construtor Python permanece público.

Fontes de implementação

api/core/patterns.py; api/ai/providers.py; api/ai/services.py

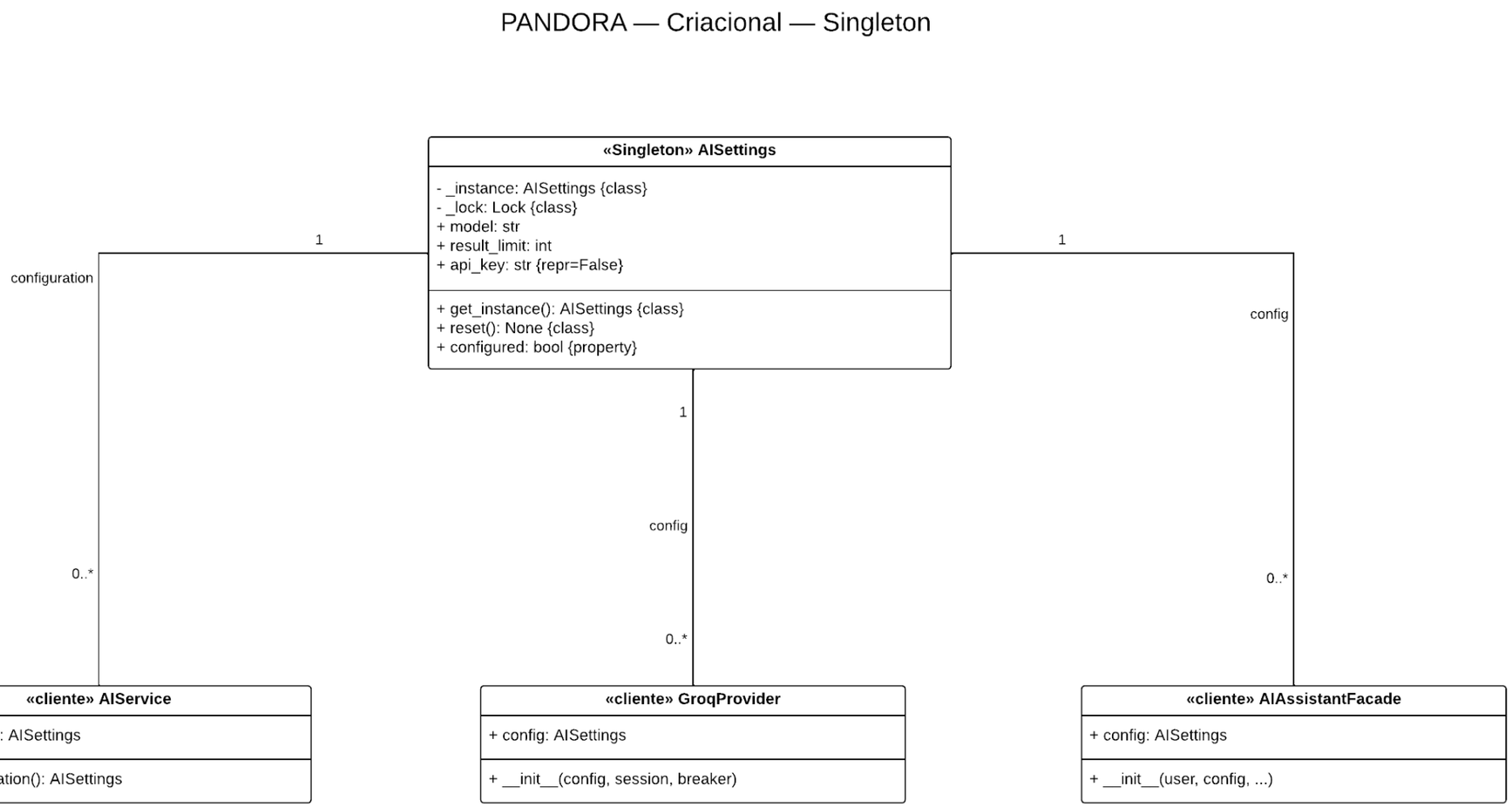


Figura 1 — Singleton. Os estereótipos identificam os papéis dos participantes.

Builder

ServiceOrderBuilderContract define as etapas de construção; ServiceOrderBuilder reúne os dados e produz ServiceOrder. ServiceOrderOpeningDirector coordena a sequência e ServiceOrderSerializer utiliza o diretor. O padrão centraliza a montagem; não substitui a validação do serializer nem libera o equipamento.

Fontes de implementação

api/maintenance/patterns.py; api/maintenance/serializers.py; api/maintenance/services.py

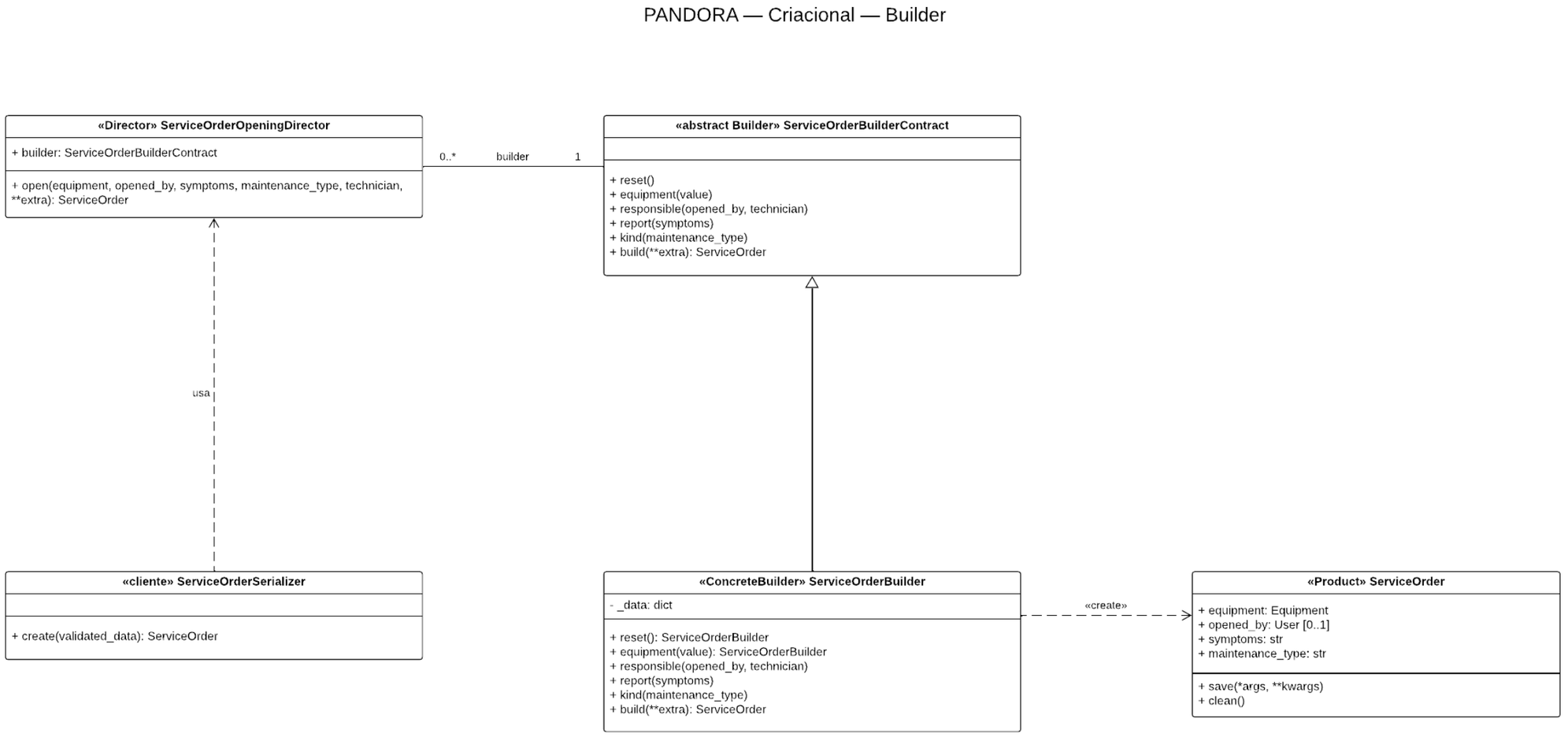


Figura 2 — Builder. Os estereótipos identificam os papéis dos participantes.

Factory Method

ReportProcessorCreator define create_processor() e o processamento pelo contrato ReportProcessor. TextReportProcessorCreator e AudioReportProcessorCreator fornecem os produtos concretos. ReportProcessorFactory seleciona o creator; essa seleção isolada não é o Factory Method. A transcrição integra o assistente.

Fontes de implementação

api/maintenance/patterns.py; api/ai/services.py

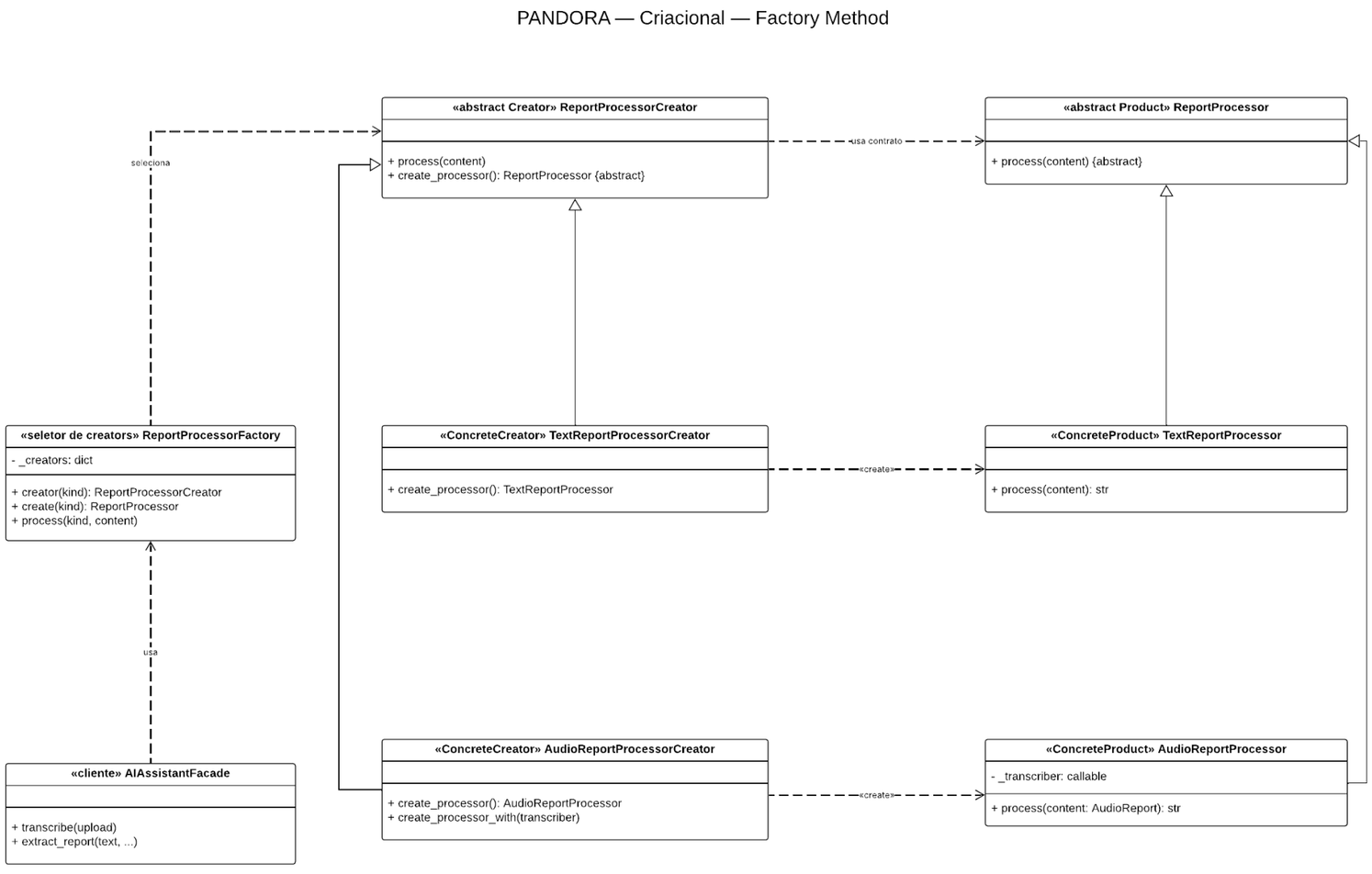


Figura 3 — Factory Method. Os estereótipos identificam os papéis dos participantes.

Abstract Factory

MaintenanceFamilyFactory define a criação de checklist e roteiro. PreventiveMaintenanceFactory e CorrectiveMaintenanceFactory criam os produtos de cada modalidade. MaintenancePlanningService utiliza a família; MaintenanceKit reúne os resultados e MaintenanceKitFactory seleciona a preparação. A finalidade é manter a coerência entre checklist e roteiro.

Fontes de implementação

api/maintenance/patterns.py; api/maintenance/views.py

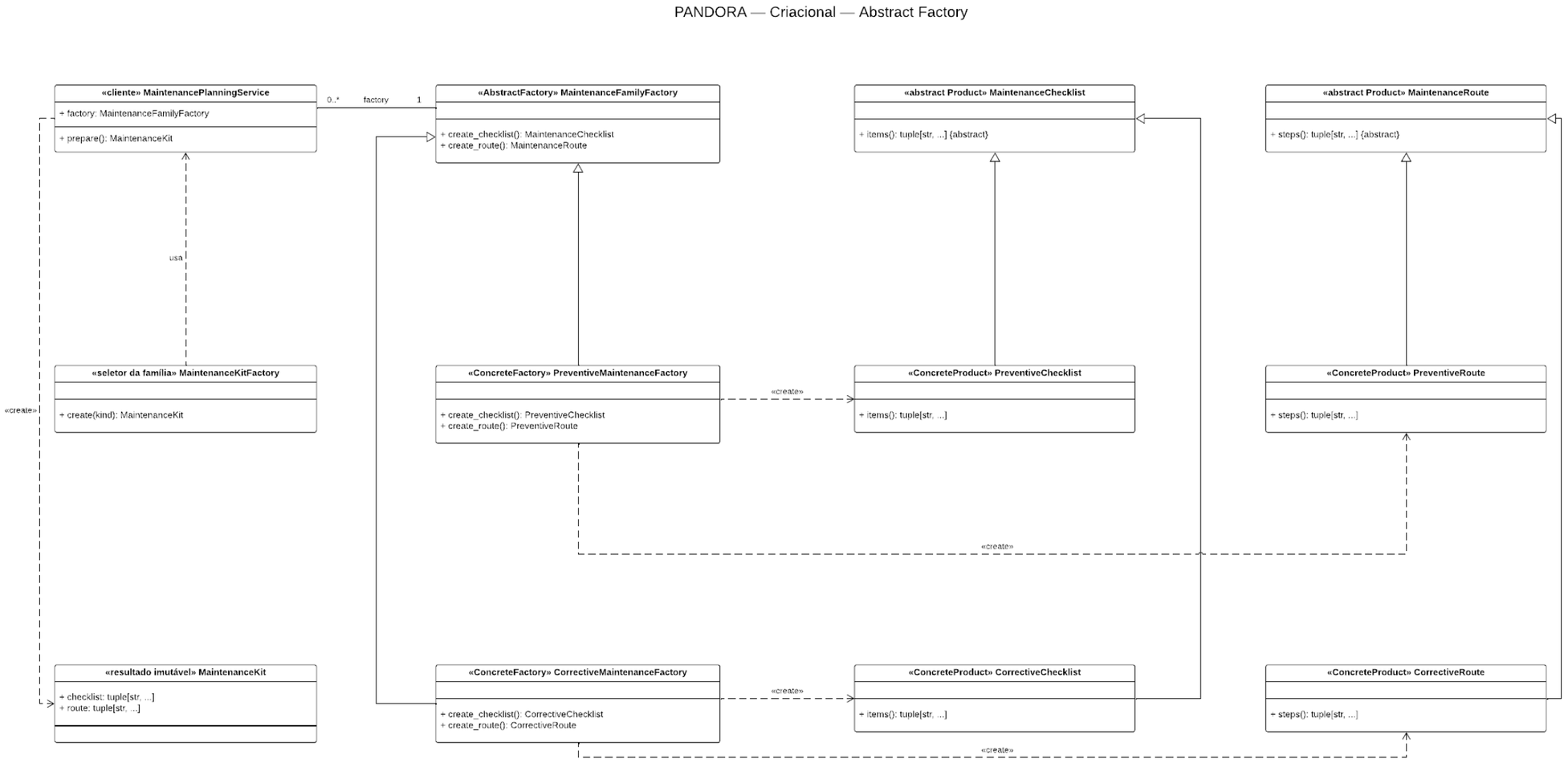


Figura 4 — Abstract Factory. Os estereótipos identificam os papéis dos participantes.

Prototype

Prototype define clone() e MaintenancePlanPrototype copia em profundidade os campos herdados da configuração do plano, permitindo informar o equipamento de destino. MaintenancePlanViewSet utiliza o clone e realiza a persistência. O objeto retornado nasce sem persistência e não replica o histórico de execução.

Fontes de implementação

api/maintenance/patterns.py; api/maintenance/views.py

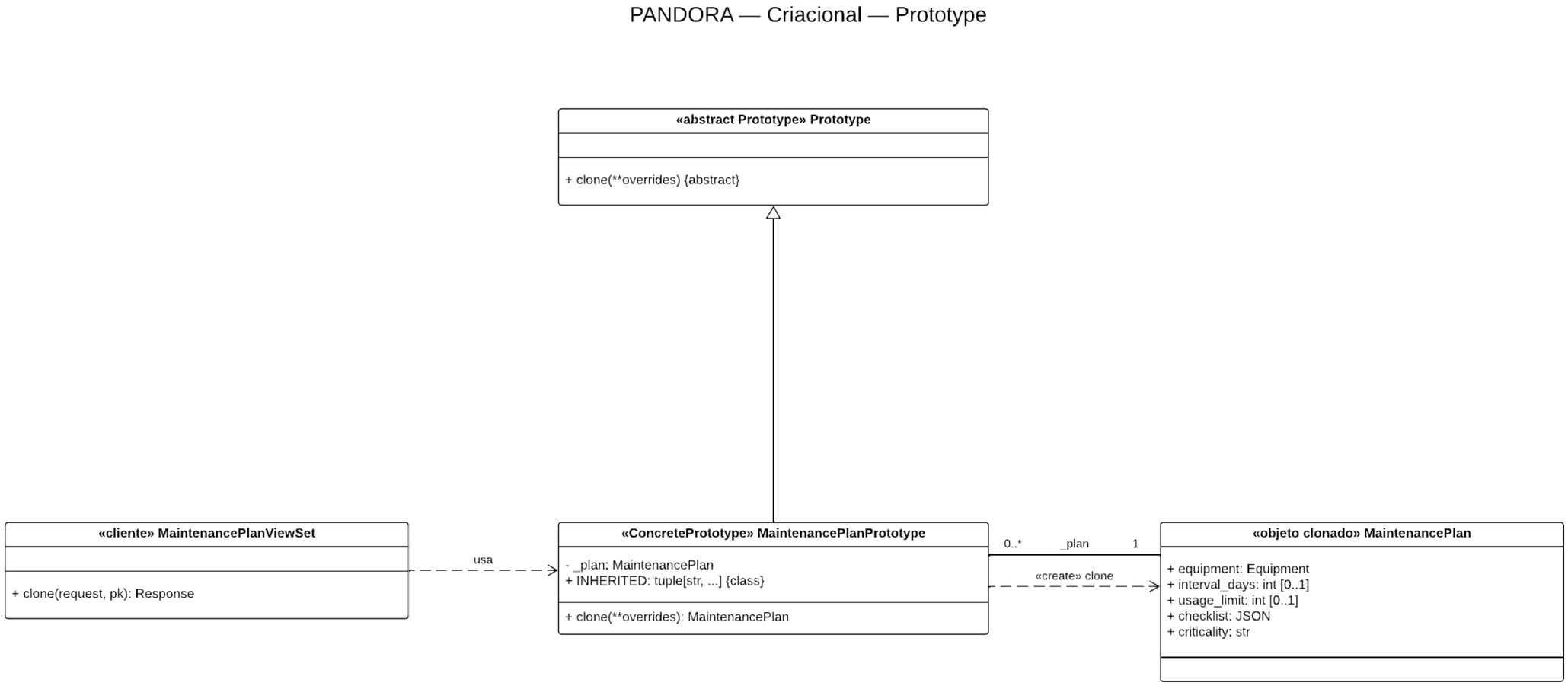


Figura 5 — Prototype. Os estereótipos identificam os papéis dos participantes.